

Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 4

Pre-Training, Training Optimizations, SFT, Evaluation, LoRA, QLoRA

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

1	Topic Map	2
2	Deep-Dive Notes	4
2.1	The LLM Training Paradigm	4
2.2	Pre-Training	4
2.2.1	Objective and Data	4
2.2.2	FLOPs vs. FLOP/s	4
2.2.3	Scaling Laws (Kaplan et al., 2020)	5
2.2.4	Chinchilla Law (Hoffmann et al., 2022)	5
2.2.5	Pre-Training Challenges	5
2.3	Training Optimizations	5
2.3.1	What Must Be Stored in GPU Memory During Training	6
2.3.2	Data Parallelism and ZeRO	6
2.3.3	Model Parallelism	6
2.3.4	FlashAttention (Dao et al., 2022)	7
2.3.5	Floating-Point Representations	8
2.3.6	Mixed Precision Training	8
2.4	Supervised Fine-Tuning (SFT)	8
2.4.1	Definition and Motivation	8
2.4.2	Instruction Tuning	9
2.4.3	Alignment	9
2.4.4	SFT Challenges	9
2.5	Evaluation of LLMs	10
2.5.1	Standard Benchmarks	10
2.5.2	Human Evaluation: Chatbot Arena / LMArena	10
2.6	Parameter-Efficient Fine-Tuning (PEFT)	10
2.6.1	LoRA: Low-Rank Adaptation (Hu et al., 2021)	11
2.6.2	QLoRA: Quantized LoRA (Dettmers et al., 2023)	12
3	Related Concepts You Should Also Know	13
4	Build Projects	14
5	Explain It Back	15
6	Interview Practice Questions & Answers	16
7	Self-Assessment Test	18

Topic Map

1. LLM Training Paradigm

- 1.1 Transfer learning vs. training from scratch
- 1.2 Two-stage pipeline: pre-training $\rightarrow$ tuning

2. Pre-Training

- 2.1 Objective: next-token prediction
- 2.2 Data mixtures: Common Crawl, Wikipedia, GitHub, StackOverflow
- 2.3 Scale: trillions of tokens; GPT-3 (300B), LLaMA 3 (15T)
- 2.4 FLOPs vs. FLOP/s notation
- 2.5 Scaling laws (Kaplan et al., 2020): bigger model, more data, more compute = better
- 2.6 Sample efficiency: bigger models learn faster per token
- 2.7 Chinchilla law (Hoffmann et al., 2022): optimal tokens $\approx 20\times$ parameters
- 2.8 Challenges: cost, knowledge cutoff, knowledge editing, plagiarism

3. Training Optimizations

- 3.1 Memory bottleneck: activations, gradients, optimizer states, parameters
- 3.2 Adam optimizer: first and second moments
- 3.3 Data parallelism: dividing batches across GPUs
- 3.4 ZeRO (Zero Redundancy Optimization): ZeRO-1, -2, -3
- 3.5 Model parallelism: tensor, pipeline, expert, sequence, context
- 3.6 FlashAttention: HBM vs. SRAM, tiling, softmax decomposition trick
- 3.7 FlashAttention backward: activation recomputation
- 3.8 Floating-point formats: FP32, FP16, BF16, FP64
- 3.9 Mixed precision training: low-precision forward/backward, high-precision weight update

4. Supervised Fine-Tuning (SFT)

- 4.1 SFT definition and objective (loss only on output, not input)
- 4.2 Instruction tuning as special case
- 4.3 Data mixture: assistant dialogues, math/reasoning/code, safety
- 4.4 Data scale: thousands to millions of examples (vs. trillions of tokens for pre-training)
- 4.5 Alignment: SFT + preference tuning
- 4.6 Mid-training (emerging concept)

5. Evaluation

- 5.1 Standard benchmarks: MMLU, ARC-Challenge, GSM8K, HumanEval
- 5.2 “Training on the test task” confound

5.3 Chatbot Arena / LMArena: A/B testing, ELO-style ranking

5.4 Limitations of all evaluation methods

6. Parameter-Efficient Fine-Tuning (PEFT)

6.1 LoRA: low-rank decomposition, $W = W_0 + BA$, rank r

6.2 LoRA placement: attention originally, FFN is most impactful

6.3 LoRA training dynamics: higher LR, poor large-batch behavior

6.4 Task-switching via LoRA adapter swap

6.5 QLoRA: quantized W_0 (NF4) + LoRA in BF16

6.6 NF4: normal-quantile-based bucketing

6.7 Double quantization: quantize the quantization constants

Deep-Dive Notes

The LLM Training Paradigm

Transfer Learning

Instead of training a model from scratch for each task, start from a model pre-trained on vast data, then adapt its weights for the target task. The pre-trained model encodes general knowledge that transfers across tasks, dramatically reducing data and compute requirements for the downstream task.

The LLM pipeline has two stages:

1. **Pre-training:** Train on massive unlabeled text to learn general language/code patterns.
2. **Tuning:** Adapt the pre-trained model for specific tasks (SFT, then preference tuning).

Pre-Training

Objective and Data

Pre-Training Objective

Train a decoder-only Transformer on next-token prediction over enormous unlabeled corpora. The model receives a sequence of tokens and must predict the next token at each position (teacher forcing during training). Loss is computed over all positions.

Data mixtures: Web-scraped text (Common Crawl: ~ 3 billion pages/month), Wikipedia, books, code (GitHub, StackOverflow), multilingual content, scientific papers. The goal is to expose the model to every pattern of language and code that exists.

Table 1: Pre-Training Scale Examples

Model	Parameters	Training Tokens
GPT-3	175B	300 billion
LLaMA 3	Various	15 trillion

FLOPs vs. FLOP/s

FLOPs and FLOP/s

- **FLOPs** (floating-point operations): a measure of *total compute* required. Training a large LLM requires $\sim 10^{25}$ FLOPs. Roughly proportional to $\text{params} \times \text{tokens}$.
- **FLOP/s** (floating-point operations per second): a measure of *hardware speed*. An H100 GPU delivers ~ 989 TFLOP/s at FP16. More FLOP/s = faster training.

Memory aid: FLOPs = “how much work?” FLOP/s = “how fast can we work?”

Notation Confusion

The literature sometimes uses “FLOPs” for both. Always contextualize: if the sentence is about a training run or cost, it means total FLOPs. If describing GPU specs, it means FLOP/s.

Scaling Laws (Kaplan et al., 2020)

Neural Scaling Laws

Model performance (measured by loss on next-token prediction) follows power-law relationships with:

- **Model size** (parameters N): larger models achieve lower loss.
- **Training data size** (tokens D): more data achieves lower loss.
- **Compute budget** (FLOPs C): more compute achieves lower loss.

These relationships hold smoothly over many orders of magnitude. “Bigger is better” drove rapid scaling from 2019–2024.

Sample Efficiency

Larger models require *fewer tokens* to reach a given loss level. For the same number of training tokens, a larger model achieves better performance than a smaller one.

Chinchilla Law (Hoffmann et al., 2022)

Chinchilla Optimal Compute Allocation

Given a fixed compute budget C , the optimal allocation is:

$$D_{\text{optimal}} \approx 20 \times N_{\text{optimal}}$$

where N = parameters and D = training tokens. In other words, **for every parameter, train on ~20 tokens**.

Implication: GPT-3 (175B parameters, 300B tokens) was *undertrained* — optimal would require ~3.5 trillion tokens. Models should be smaller but trained longer to reach the same compute-optimality.

How Companies Use Chinchilla

In practice, companies run scaling law experiments on smaller models (cheap) and extrapolate. LLaMA 3 includes an analysis justifying its parameter/token choices. The ratio also depends on architecture (MoE uses fewer active FLOPs per token) and precision.

Pre-Training Challenges

- **Cost:** Minimum millions of dollars; frontier models cost hundreds of millions. Environmental impact is significant.
- **Knowledge cutoff:** The model knows only about events up to the data collection date. No way to know post-cutoff events from weights alone.
- **Knowledge editing:** Injecting new knowledge into weights without degrading other capabilities is an open research problem.
- **Plagiarism/memorization:** Models can reproduce training data verbatim, raising copyright concerns.

Training Optimizations

What Must Be Stored in GPU Memory During Training

A single forward+backward pass requires storing:

1. **Model parameters** (W): FP32 = 4 bytes/param; for 70B params $\approx$ 280 GB.
2. **Activations**: Intermediate layer outputs needed for gradient computation. Size $\propto$ model depth $\times$ batch size $\times$ sequence length.
3. **Gradients** (∇W): Same size as parameters.
4. **Optimizer states**: Adam stores first moment m and second moment v per parameter — $2\times$ the parameter count, also in FP32.

Adam Optimizer Memory

Adam maintains:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment, EMA of gradients})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment, EMA of squared gradients})$$

$$W_{t+1} = W_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

For 70B-param model in FP32: parameters (280 GB) + gradients (280 GB) + optimizer states (560 GB) $\approx$ 1.1 TB. A single H100 has 80 GB — you need at least 14 GPUs just for this.

Data Parallelism and ZeRO

Data Parallelism (DP)

Divide each training batch across K GPUs. Each GPU holds a *full copy* of the model and processes its local mini-batch independently. Gradients are averaged across GPUs after the backward pass (all-reduce communication). Memory per GPU is unchanged; throughput scales with K .

ZeRO: Zero Redundancy Optimization (Rajbhandari et al., 2019)

Eliminates redundant storage across GPUs by sharding the model state across K devices:

- **ZeRO-1**: Shard optimizer states only. Memory $\approx 4\times$ reduction at K GPUs.
- **ZeRO-2**: Shard optimizer states + gradients. Memory $\approx 8\times$ reduction.
- **ZeRO-3**: Shard optimizer states + gradients + parameters. Memory $\approx K$ -fold reduction. Each GPU holds $1/K$ of the model; parameters are gathered on demand.

Trade-off: each ZeRO stage incurs more inter-GPU communication (all-gather, reduce-scatter) to reconstruct sharded tensors during forward/backward.

Model Parallelism

Model Parallelism

Splits the model itself (not just the data) across GPUs. Key variants:

- **Tensor Parallelism (TP)**: Split large matrix multiplications across GPUs (e.g., Megatron-LM splits attention heads across devices).
- **Pipeline Parallelism (PP)**: Assign different layers to different GPUs. GPU 1 handles

layers 1–8, GPU 2 handles layers 9–16, etc. Micro-batching reduces “pipeline bubbles.”

- **Expert Parallelism (EP):** In MoE models, place each expert on a separate GPU. Tokens are routed to the correct GPU.
- **Sequence/Context Parallelism:** Split the sequence dimension across GPUs for very long contexts.

In practice, large-scale training combines DP + TP + PP (3D parallelism). Reference: HuggingFace’s “Ultra-Scale Playbook.”

FlashAttention (Dao et al., 2022)

GPU Memory Hierarchy

- **HBM (High-Bandwidth Memory):** Large (~ 80 GB on H100) but relatively slow ($\sim 2\text{--}3$ TB/s). Persistent GPU memory.
- **SRAM (on-chip):** Small ($\sim 20\text{--}40$ MB on H100) but very fast ($\sim 20\text{--}40$ TB/s). Located next to the compute units (CUDA cores / tensor cores).

Standard attention is memory-bandwidth-bound: the bottleneck is reading/writing Q, K, V, S, P, O to/from HBM, not the FLOP count itself.

Standard Attention Memory Access Pattern

For inputs $Q, K \in \mathbb{R}^{n \times d_k}$ and $V \in \mathbb{R}^{n \times d_v}$:

1. Load Q, K from HBM $\rightarrow$ compute $S = QK^\top / \sqrt{d_k} \rightarrow$ write $S \in \mathbb{R}^{n \times n}$ to HBM.
2. Load S from HBM $\rightarrow$ compute $P = \text{softmax}(S) \rightarrow$ write P to HBM.
3. Load P, V from HBM $\rightarrow$ compute $O = PV \rightarrow$ write O to HBM.

Total HBM accesses: $O(n^2)$. For $n = 4096$, this is the bottleneck.

FlashAttention: Tiling and Online Softmax

Key idea: Process attention in blocks that fit in SRAM. Avoid materializing S and P in HBM entirely.

Tiling: Divide Q, K, V into blocks. For each pair of Q -block and K -block, compute the attention contribution and accumulate into O , then move to the next block. The full $n \times n$ matrix S is never written to HBM.

Online softmax trick: Softmax is row-wise: $P_{ij} = \exp(S_{ij}) / \sum_k \exp(S_{ik})$. The denominator requires seeing the full row. However, softmax can be computed incrementally:

$$\text{softmax}([S_1, S_2]) = \frac{\exp(S_1) + \exp(S_2)}{\exp(m) \cdot \text{const}}$$

where $m = \max(S_1, S_2)$ is a running maximum. Each block adjusts the accumulated output with a rescaling factor, producing the exact same result as standard attention.

Result: HBM accesses reduced from $O(n^2)$ to $O(n)$. Runtime reduced $\sim 3\text{--}8\times$ for typical sequence lengths.

FlashAttention Backward Pass: Activation Recomputation

Standard backward pass stores the attention matrix $P \in \mathbb{R}^{n \times n}$ from the forward pass to compute gradients. FlashAttention recomputes P during the backward pass using the same tiling trick rather than storing it.

Counterintuitive result: Recomputing costs more FLOPs, but the backward pass is still

faster because the HBM bandwidth savings dominate. Both memory *and* runtime improve simultaneously.

Floating-Point Representations

Table 2: Floating-Point Format Comparison

Format	Sign	Exponent	Mantissa	Notes
FP64	1	11	52	Maximum precision; very slow on GPU
FP32	1	8	23	Standard training precision
BF16	1	8	7	Same range as FP32; preferred over FP16 for training
FP16	1	5	10	Smaller range; risk of overflow

Why BF16 over FP16 for Training?

BF16 has the same 8-bit exponent as FP32, giving it the same *dynamic range*. This makes overflow/underflow rare during training. FP16's smaller exponent (5 bits) makes it prone to numerical instability. BF16 mantissa (7 bits) is less precise than FP16 (10 bits), but this matters less for training than dynamic range. Most modern LLM training uses BF16.

Mixed Precision Training

Mixed Precision Training (Micikevicius et al., 2017)

Use different precisions for different operations:

- **Forward pass:** Activations in FP16/BF16. Faster GEMM operations; lower memory.
- **Backward pass:** Gradient updates in FP16/BF16.
- **Weight update:** Master weights stored in FP32. The update $W \leftarrow W + \alpha g$ is applied in FP32.

Why keep weights in FP32? Accumulated gradient updates can be very small. Low-precision arithmetic would cause updates below the minimum representable value to be rounded to zero, halting learning.

H100 compute speed: FP64 = 34 TFLOP/s, FP32 = 67 TFLOP/s, FP16/BF16 = 989 TFLOP/s (with sparsity). Lower precision $\approx 15\times$ speedup.

Supervised Fine-Tuning (SFT)

Definition and Motivation

Supervised Fine-Tuning (SFT)

After pre-training, the model generates plausible text continuations but is not helpful as an assistant — it completes prompts rather than answering them. SFT trains the model on labeled (input, output) pairs to change its behavior.

Objective: Next-token prediction, but the loss is computed *only on the output tokens*, not

the input. The input is given (conditioned on), not predicted.

SFT Loss Masking

Input prompt: [BOS] Can I put my teddy bear in the washer?

Target output: No, it might get damaged. Try hand washing instead.

During SFT, loss is computed only on the output tokens (green). The input tokens are not used in the loss calculation — we don't want the model to “learn to ask the question,” only to generate the answer given the question.

Instruction Tuning

Instruction Tuning

A special case of SFT where the training data consists of (instruction, response) pairs that teach the model to follow natural language instructions. This “graduates” the pre-trained model to being a helpful assistant. Key data categories:

- **Assistant dialogues:** Story writing, list generation, explanation, Q&A.
- **Synthetic instructions:** Generated by existing LLMs, reviewed by humans.
- **Math, reasoning, code:** High-quality problem-solution pairs.
- **Safety alignment:** Examples of refusals, hedging, and harm avoidance.

Table 3: SFT Data Scale vs. Pre-Training

Model	Pre-Training	SFT Examples	Approx. SFT Tokens
GPT-3	300B tokens	13K examples	~13M tokens
LLaMA 3	15T tokens	10M examples	~10B tokens

SFT data is **4–5 orders of magnitude smaller** than pre-training data, but must be **very high quality**. Pre-training teaches language; SFT teaches behavior.

Alignment

Alignment

The process of making an LLM's behavior consistent with human values and intentions. Consists of two stages:

1. **SFT (Supervised Fine-Tuning):** Teach helpful, harmless behavior via labeled examples.
2. **Preference Tuning (e.g., RLHF/DPO):** Further shape behavior using human preference signals. Covered in Lecture 5.

An emerging additional stage called **mid-training** (between pre-training and SFT) uses the pre-training objective but on domain-specific, higher-quality data to bridge the gap.

SFT Challenges

- **Data quality:** High-quality SFT data requires significant human effort or strong LLM-generated + human-reviewed pipelines.
- **Prompt distribution sensitivity:** The model generalizes well to prompts similar to training distribution but may struggle with out-of-distribution queries.

- **Generalization:** The model must learn the concept (e.g., “write a story”) not just memorize examples.
- **Evaluation difficulty:** Subjective quality is hard to quantify.
- **Computational cost:** Fine-tuning even a medium-sized model requires significant GPU resources.

Evaluation of LLMs

Standard Benchmarks

Table 4: Major LLM Evaluation Benchmarks

Benchmark	What It Measures	Format
MMLU	General knowledge (57 subjects)	Multiple choice
ARC-Challenge	Grade-school science reasoning	Multiple choice
GSM8K	Grade-school math word problems	Open-ended generation
HumanEval	Code generation	Pass@ k on test cases

The “Training on the Test Task” Confound

If a model is trained on data from the same *domain* as a benchmark (e.g., math reasoning data for GSM8K), it will score higher regardless of general capability improvement. Benchmark scores are only comparable across models if both were either trained on the test task or both were not. (Dominguez-Olmedo et al., 2024)

Human Evaluation: Chatbot Arena / LMArena

Chatbot Arena

A crowd-sourced evaluation platform where users submit real queries and are shown two anonymous model responses (A/B test). They vote for the better response. ELO-style ranking aggregates pairwise comparisons into a leaderboard.

Benefits: Reflects real user experience; not tied to specific benchmark formats; covers a broad distribution of prompts.

Limitations of Chatbot Arena

- **Cold start problem:** New models are paired with few other models initially; early matchups disproportionately influence final ELO.
- **Riggable:** An adversarial player could detect which model they’re rating (e.g., by asking “who are you?”) and selectively upvote specific models.
- **Factuality blind spot:** Users often cannot assess factual accuracy for complex topics.
- **Non-representative raters:** The voting population (technical users, enthusiasts) differs from the general user base.
- **Safety penalization:** Users tend to prefer responses over refusals, even when refusal is the correct behavior.

Takeaway: No single evaluation method captures all dimensions. Use multiple: benchmarks + human eval + domain-specific testing.

Parameter-Efficient Fine-Tuning (PEFT)

LoRA: Low-Rank Adaptation (Hu et al., 2021)**LoRA**

Instead of updating the full weight matrix $W \in \mathbb{R}^{d \times k}$ during fine-tuning, decompose the update as a product of two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + BA$$

where $W_0 \in \mathbb{R}^{d \times k}$ is the **frozen** pre-trained weight, $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are the **trained** LoRA matrices, and $r \ll \min(d, k)$ is the **rank** (typically 4–64).

During the forward pass:

$$\mathbf{h} = W_0 \mathbf{x} + BA \mathbf{x} = W_0 \mathbf{x} + \Delta W \mathbf{x}$$

Only B and A require gradients. W_0 gradients are never computed.

Parameter Count Reduction

Full fine-tuning trains $d \times k$ parameters. LoRA trains $r \times d + r \times k = r(d + k)$ parameters.

Example: $d = k = 4096$, $r = 8$:

- Full: $4096^2 = 16.7\text{M}$ parameters
- LoRA: $8 \times (4096 + 4096) = 65.5\text{K}$ parameters
- Reduction: $\approx 250\times$

Task Switching with LoRA

The pre-trained W_0 is shared across tasks. For each task, train a separate (B, A) pair. At inference time, load W_0 once and swap LoRA adapters to switch tasks instantly. This enables efficient multi-task serving from a single base model.

Where to Apply LoRA

- **Original paper:** Applied to attention projection matrices (W_Q, W_K, W_V, W_O).
- **Current guidance** (“LoRA Without Regret,” Schulman et al., 2025): FFN blocks provide the most performance improvement per added parameter. In practice, apply LoRA to both attention and FFN, with most impact in FFN.

LoRA Training Dynamics (empirical)

- **Higher learning rate needed:** LoRA converges better with $\sim 10\times$ higher LR than full fine-tuning. Hypothesis: the low-rank constraint forces the optimizer to explore a restricted subspace that requires larger steps.
- **Large batch size hurts:** LoRA underperforms full fine-tuning at large batch sizes. Hypothesis: the training dynamics of matrix products BA differ from single matrices; larger batches may flatten useful gradient directions.

QLoRA: Quantized LoRA (Dettmers et al., 2023)

QLoRA

Combine quantization of W_0 with LoRA fine-tuning of B, A :

- W_0 : Quantized to 4-bit NF4 format (stored quantized, dramatically reducing VRAM).
- B, A : Stored and computed in BF16 (full precision).
- Forward pass computation: dequantize W_0 to BF16 on the fly, compute $W_0\mathbf{x}$, add $BA\mathbf{x}$.

Result: enables fine-tuning of LLaMA 65B on a single 48GB GPU; reported $\sim 16\times$ VRAM savings.

NF4: 4-bit NormalFloat

Standard INT8 quantization divides the numeric range into **uniform buckets**. NF4 instead uses **quantile-based bucketing** based on the assumption that weight values follow a normal distribution.

For a normal distribution, quantile-based buckets place roughly equal numbers of values in each bin, minimizing quantization error (since most values cluster near zero). This is more efficient than uniform buckets where the outlier regions waste precision.

Double Quantization

Standard quantization requires per-block **quantization constants** (e.g., scale factors) that must be stored in full precision. QLoRA applies a second round of quantization to these constants themselves, saving an additional $\sim 6\%$ memory. The effect is small but meaningful at scale.

Related Concepts You Should Also Know

1. **AdamW (Loshchilov & Hutter, 2017):** The standard variant of Adam used for LLM training. Adds decoupled weight decay: rather than including weight decay in the gradient update (which interacts poorly with Adam’s normalization), it applies weight decay directly to the weights after the Adam step. Helps generalization and prevents weight explosion.
2. **Learning Rate Schedule:** LLM training typically uses a **warmup** phase (linear increase from 0 to peak LR over the first few thousand steps) followed by **cosine decay**. Warmup prevents early instability from large gradient magnitudes at initialization.
3. **Gradient Clipping:** Cap the gradient norm to a maximum value (e.g., 1.0) to prevent exploding gradients during training. Especially important for long sequences and deep models.
4. **Epoch vs. Token Count:** LLM pre-training is typically described in *tokens processed*, not epochs. The training set is so large that the model may see each document only once (“epoch ≈ 1 ”). This is fundamentally different from supervised ML where multiple epochs are standard.
5. **FSDP (Fully Sharded Data Parallel):** PyTorch’s implementation of ZeRO-3-like sharding. Standard for training large models in PyTorch. DeepSpeed is another common framework that implements ZeRO.
6. **Prefix Tuning and Adapters:** Two other PEFT methods mentioned briefly in the lecture. Prefix tuning prepends trainable vectors to the attention key/value sequences. Adapters insert small bottleneck layers between Transformer sublayers. Both are less commonly used than LoRA in practice.
7. **LoRA Rank Selection:** Common values: $r \in \{4, 8, 16, 32\}$. Higher rank = more parameters = more expressive but more expensive. For most tasks, $r = 8$ or $r = 16$ is a good starting point. Grid search is possible but practitioners often rely on community benchmarks.
8. **MMLU (Massive Multitask Language Understanding):** 57 subject areas (STEM, humanities, social sciences) with 4-choice questions. A gold standard for general knowledge evaluation. Score of 25% = random. Human expert $\approx 89.8\%$. GPT-4 $\approx 86.4\%$.
9. **Mid-Training (emerging):** A training stage between pre-training and SFT that uses the pre-training objective (next-token prediction) but on curated, domain-specific data. Bridges the gap between general language modeling and task-specific behavior without the label cost of SFT.
10. **Synthetic Data Generation:** Using existing strong LLMs (e.g., GPT-4) to generate instruction-response pairs for SFT. Reduces human labeling cost but can introduce distribution shifts (e.g., “model collapse” if the student model trains on its own outputs). Human review of synthetic data is important for quality control.

Build Projects

1. **LoRA Fine-Tuning Pipeline** (*5–8 hours, very high signal*)

This maps directly to your Finetune-Pipeline work. Using HuggingFace PEFT + SFT-Trainer, fine-tune a 7B model (Mistral or LLaMA 3-8B) on a task of your choice (e.g., domain-specific QA, classification, code generation). Experiment with ranks $r \in \{4, 8, 16\}$ and learning rates. Compare LoRA vs. full fine-tuning on a held-out eval set. Write up the memory savings (GPU VRAM usage at each rank), training time, and quality trade-off. This is an ideal portfolio piece for Applied Scientist roles.

2. **QLoRA Experiment on a 13B+ Model** (*4–6 hours, high signal*)

Apply QLoRA to fine-tune LLaMA 3-13B or a similar model on a consumer GPU (e.g., RTX 4090, 24GB VRAM, or Colab A100). Profile: peak VRAM before and after QLoRA, training throughput (tokens/second), final loss curve. Verify that the $16\times$ VRAM savings are approximately achieved. This is a direct demonstration of the concepts from this lecture.

3. **FlashAttention Benchmark** (*2–3 hours, good pedagogical signal*)

Implement a benchmark that compares standard PyTorch attention vs. FlashAttention (available in `torch.nn.functional.scaled_dot_product_attention` with the flash backend) for sequence lengths $n \in \{512, 1024, 2048, 4096, 8192, 16384\}$. Measure: peak GPU memory (VRAM), forward-pass time, backward-pass time. Plot the scaling curves. Verify the $O(n)$ memory behavior of FlashAttention vs. $O(n^2)$ for standard attention.

Explain It Back

1. A senior engineer says: “Just scale up the model to 1 trillion parameters and train it for a week — it’ll be great.” Using the Chinchilla law, explain why this may not be compute-optimal. What should they do instead? Give a concrete example with numbers.
2. You are presenting to a non-technical stakeholder who asks why GPT-3 is “undertrained.” Explain the Chinchilla law in plain language (no equations), using the analogy of study time vs. number of textbooks.
3. Explain the FlashAttention tiling trick to a software engineer who knows matrix operations but doesn’t know ML. Focus on the HBM vs. SRAM distinction and why minimizing HBM reads/writes matters more than minimizing FLOPs. Use concrete numbers from the H100 spec.
4. Your team is debating whether to use full fine-tuning or LoRA for adapting LLaMA 3-70B to your company’s internal domain. Walk through the trade-offs, covering: parameter count, VRAM requirements, training time, multi-task serving, and any empirical caveats about LoRA’s training dynamics.

Interview Practice Questions & Answers

L1 — What is the Chinchilla law and what does it say about GPT-3?

Answer: The Chinchilla law (Hoffmann et al., 2022) states that for compute-optimal training, the number of training tokens should be approximately $20\times$ the number of model parameters. For GPT-3 (175B parameters), the optimal token count would be ~ 3.5 trillion, but GPT-3 was only trained on 300 billion tokens. This means GPT-3 was severely undertrained relative to its parameter count — the same compute budget could achieve better performance with a smaller model trained on more data. LLaMA 3 applied this insight, training a range of models on 15 trillion tokens.

L1 — What is the difference between SFT and pre-training in terms of data and objective?

Answer: Pre-training uses raw, unlabeled internet text (trillions of tokens) and trains on next-token prediction over the full sequence. SFT uses labeled (instruction, response) pairs (thousands to millions of examples, orders of magnitude less data). The key objective difference: in pre-training, loss is computed over all token positions. In SFT, the loss is masked to apply only to the output/response tokens — the input prompt is conditioned on but not predicted. This prevents the model from “learning to ask questions” and focuses learning on the response generation.

L2 — Explain ZeRO stages and when you would use each.

Answer: ZeRO eliminates redundant storage in data-parallel training by sharding model state across GPUs. ZeRO-1 shards optimizer states (Adam’s m and v): $\sim 4\times$ memory reduction, minimal communication overhead. ZeRO-2 additionally shards gradients: $\sim 8\times$ reduction, moderate overhead. ZeRO-3 also shards parameters: $K\times$ reduction for K GPUs, but requires all-gather of parameters during forward/backward passes — highest communication cost. Use ZeRO-1 when model fits on one GPU but optimizer states don’t. ZeRO-2 when gradients also overflow. ZeRO-3 when even the model doesn’t fit on a single GPU.

L2 — Why does FlashAttention recomputing activations in the backward pass result in faster training, not slower?

Answer: Counterintuitively, recomputing attention activations during the backward pass is faster than storing and reloading them, because LLM training is memory-bandwidth-bound, not compute-bound. Standard attention stores the full $n \times n$ attention matrix P in HBM after the forward pass, then reloads it during the backward pass. This requires $O(n^2)$ HBM reads and writes. FlashAttention recomputes P using SRAM-resident tiling, avoiding those HBM transfers entirely. Even though more floating-point operations are executed (higher FLOPs), the elimination of slow HBM reads/writes dominates, yielding both lower latency and lower peak memory.

L2 — Explain LoRA’s low-rank decomposition. Why does it reduce memory?

Answer: LoRA decomposes the weight update as $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$. The frozen pre-trained weight W_0 requires no gradients, so no gradient tensors are allocated for it. Only B and A require gradients and optimizer states. For a typical attention matrix with $d = k = 4096$ and $r = 8$: LoRA trains 65K parameters vs. 16.7M for full fine-tuning — a $256\times$ reduction. Since Adam optimizer states are $2\times$ the

parameter size in FP32, the optimizer memory reduction is proportional.

L3 — Describe the QLoRA approach and why NF4 is better than uniform INT8 for quantizing LLM weights.

Answer: QLoRA quantizes the frozen base model weights W_0 to 4-bit NF4 while keeping the LoRA matrices B, A in BF16. During the forward pass, W_0 is dequantized to BF16 on the fly for computation. Standard INT8 quantization divides the numeric range into equal-width buckets. However, LLM weights approximately follow a normal distribution (concentrated near zero with thin tails). Uniform buckets waste bit-resolution on the sparse tail regions while under-representing the dense central region. NF4 uses quantile-based buckets derived from the normal distribution's CDF, placing equal numbers of values into each bucket. This minimizes expected quantization error for normally-distributed values, achieving better quality at the same bit-width.

L3 — What are the practical limitations of benchmark-based LLM evaluation?

Answer: Three main issues. First, **training on the test task**: a model trained on math-reasoning data will outperform on GSM8K regardless of general reasoning improvement. Benchmarks are only comparable if both models have or have not been trained on the test task's domain. Second, **benchmark saturation**: once top models approach human performance on a benchmark, it no longer discriminates between models. New benchmarks are needed but themselves may be optimized for shortly after release. Third, **capability vs. usefulness gap**: a model can score highly on MMLU while being unhelpful in practice due to poor instruction-following, verbosity, or refusing valid requests. Human evaluation (Chatbot Arena) partially addresses this but has its own issues: cold-start bias, riggability, rater non-representativeness, and factuality blindness.

L1 — Why does mixed precision training keep weights in FP32 but use FP16/BF16 for activations?

Answer: Activations are intermediate values computed during the forward pass on a specific batch of data. They are noisy and transient — full FP32 precision is wasteful. Weights, however, accumulate tiny gradient updates across many training steps. If weights are stored in FP16, the update ΔW may be smaller than the minimum representable increment in FP16, causing the update to round to zero and learning to stall. FP32 weights have ~ 7 decimal digits of precision vs. FP16's ~ 3 , ensuring small but meaningful updates are preserved. The speedup comes from FP16 activations enabling faster GEMM operations; the quality is preserved by FP32 weight masters.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

- Q1.** According to the Chinchilla law, for a model with 10 billion parameters, the compute-optimal training token count is approximately:
 (a) 10 billion (b) 100 billion (c) 200 billion (d) 10 trillion
- Q2.** In ZeRO-3, which of the following is sharded across GPUs?
 (a) Only optimizer states (b) Optimizer states and gradients (c) Optimizer states, gradients, and parameters (d) Only parameters
- Q3.** FlashAttention achieves its speedup primarily by:
 (a) Reducing the total number of FLOPs (b) Minimizing read/write operations to HBM using SRAM tiling (c) Approximating the attention matrix with a low-rank representation (d) Using INT8 quantization for the attention computation
- Q4.** In SFT, the training loss is computed over:
 (a) All tokens (input + output) (b) Only the input/prompt tokens (c) Only the output/response tokens (d) A random 15% of all tokens (like BERT’s MLM)
- Q5.** In LoRA with rank $r = 4$ applied to a 1024×1024 weight matrix, how many trainable parameters are introduced?
 (a) $4 \times 1024 = 4096$ (b) $4 \times (1024 + 1024) = 8192$ (c) $4 \times 1024 \times 2 = 8192$ (d) $1024^2/4 = 262144$
- Q6.** QLoRA stores the frozen base model weights in:
 (a) FP32 (full precision) (b) BF16 (c) FP16 (d) 4-bit NF4
- Q7.** BF16 is preferred over FP16 for LLM training because:
 (a) BF16 has higher mantissa precision (b) BF16 has the same dynamic range as FP32 (same exponent size) (c) BF16 is supported on more hardware (d) BF16 reduces memory by $4\times$ vs. FP32
- Q8.** Which component of GPU memory is fast but small?
 (a) HBM (b) DRAM (c) SRAM (d) VRAM (same as HBM)

Short Answer (1.5 points each)

- Q9.** Name the four types of quantities that must be stored in GPU memory during LLM training and explain which is unique to Adam (not SGD).
- Q10.** What is the “knowledge cutoff date” for an LLM and why is it a limitation?
- Q11.** Explain in one paragraph why the Chatbot Arena “cold start problem” makes ELO rankings unreliable for newly released models.
- Q12.** What is the benefit of LoRA’s task-switching property compared to full fine-tuning for a multi-task serving setup?
- Q13.** What does “sample efficiency” mean in the context of scaling laws? Which is more sample-efficient: a 7B or a 70B parameter model?
- Q14.** Describe the online softmax trick that enables FlashAttention to compute attention without materializing the full $n \times n$ score matrix.

Scenario (2 points)

Q15. Your startup wants to build a domain-specific LLM assistant (medical Q&A) on a budget. You have one NVIDIA A100-80GB GPU and 10K high-quality (question, answer) pairs. Describe your full training approach from the base model choice through to serving, referencing QLoRA, LoRA rank selection, SFT loss masking, and evaluation strategy. What benchmarks would you use and why?

Answer Key

Q1. (c) $20 \times 10\text{B} = 200$ billion tokens.

Q2. (c) ZeRO-3 shards all three: optimizer states, gradients, and parameters.

Q3. (b) FlashAttention’s speedup comes from minimizing slow HBM read/writes via SRAM tiling. Total FLOPs actually increase (more recomputation in backward pass).

Q4. (c) Loss is computed only on output/response tokens. Input tokens are masked out from the loss.

Q5. (b) $B \in \mathbb{R}^{1024 \times 4}$ (4,096 params) + $A \in \mathbb{R}^{4 \times 1024}$ (4,096 params) = 8,192 total trainable parameters.

Q6. (d) QLoRA stores frozen W_0 in 4-bit NF4 format. The LoRA matrices B, A are stored in BF16.

Q7. (b) BF16 uses 8 exponent bits (same as FP32), giving the same dynamic range. FP16 has only 5 exponent bits, leading to overflow/underflow during training.

Q8. (c) SRAM is small ($\sim 20\text{--}40$ MB) but very fast ($\sim 20\text{--}40$ TB/s). HBM is large (80 GB on H100) but slower (~ 3 TB/s).

Q9. (1) *Model parameters* (W), (2) *Activations* (needed for gradient computation), (3) *Gradients* (∇W), (4) *Optimizer states*. The optimizer states (first moment m and second moment v in Adam) are unique to Adam — SGD only stores parameters and gradients.

Q10. The knowledge cutoff is the date up to which pre-training data was collected. Events occurring after this date are unknown to the model from its weights alone. It’s a limitation because: (a) the model cannot answer questions about recent events, (b) the model may give outdated information confidently, (c) injecting new knowledge into existing weights without degrading other capabilities is an unsolved problem (requiring RAG, tool use, or continual pre-training as workarounds).

Q11. ELO ratings depend on the number and quality of pairwise comparisons. A newly released model starts with few matchups and is paired somewhat randomly. If those early matchups happen to be against weak or strong opponents disproportionately, the resulting ELO can be artificially inflated or deflated. Since ELO updates are cumulative and early games have high uncertainty, these initial biases can persist as the model accumulates more matches, making the final rating unreliable for models that have not yet played thousands of games.

Q12. In multi-task serving, full fine-tuning creates a separate complete copy of the model for each task (e.g., one 70B model for spam, another for sentiment). This requires enormous storage and VRAM. With LoRA, only one copy of W_0 (70B frozen) is needed, shared across all tasks. Each task requires only its small (B, A) adapter ($\sim$ millions of params vs. billions). Switching tasks means swapping the adapter — no additional GPU memory. For N tasks: full fine-tuning requires $N \times 70\text{B}$ parameter copies; LoRA requires 1 base + $N \times$ (tiny adapters).

Q13. Sample efficiency means achieving a given quality level with fewer training tokens. A 70B model is more sample-efficient than a 7B model: for the same number of tokens processed, the 70B model reaches lower loss. Intuitively, larger models can represent more complex patterns and extract more signal from each training example.

Q14. The softmax of a concatenated row $[S_1, S_2, \dots, S_k]$ can be decomposed as: compute softmax of S_1 up to a running max m_1 , then when S_2 arrives, update the running max $m_{1:2} = \max(m_1, m_2)$ and rescale the previous partial result by $\exp(m_1 - m_{1:2})$ before adding the contribution from S_2 . Repeating this block by block produces the exact final softmax. FlashAttention

uses this to process attention in tiles that fit in SRAM, accumulating the output O with rescaling factors, without ever writing the full $n \times n$ attention matrix to HBM.

Q15. Start with a pre-trained base model (LLaMA 3-8B or Mistral-7B) — small enough to fit with QLoRA on one A100. Apply QLoRA: quantize base weights to NF4 ($16\times$ VRAM reduction), add LoRA adapters at attention and FFN layers with rank $r = 16$, train in BF16. For SFT: format your 10K pairs as instruction-response; mask loss to apply only to response tokens. Use a learning rate $\sim 2 \times 10^{-4}$ (higher than full fine-tuning as per LoRA guidance), linear warmup + cosine decay. Evaluation: (1) domain-specific held-out test set (200–500 examples, human-reviewed), (2) general benchmarks MMLU to check for catastrophic forgetting, (3) MedQA if available as a medical domain benchmark. For serving: merge LoRA weights into base for faster inference, quantize inference to INT8 with `llm.int8()`, serve with vLLM for KV-cache efficiency. Track perplexity and response quality over training; if quality plateaus, consider adding more diverse medical QA data.

End of Study Guide — Lecture 4

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.